`ApplicationJob` is a handy place to put job-specific helper methods you may discover you need. You could add one to extract the code we added for transient errors in *Prevent Transient Errors from Notifying You*. A better way is to use a server middleware.

Middleware in Sidekiq is like Rack Middleware[26]—it allows you to inject code into Sidekiq's internal process. Sidekiq has *server middleware*, which allows you to run code around when a job is executed, and *client middleware*, which allows running code around when a job is queued.

By using a server middleware, you can catch every exception thrown by a job and examine it. If the exception is known to be transient, you can wrap it in a `IgnorableExceptionSinceSidekiqWillRetry`. Otherwise, you can let it bubble up as normal.

Let's build and test this middleware. The middleware itself can maintain a list of common transient exceptions (like `BaseServiceWrapper::HTTPError`), but it will also allow job classes to specify their own list of transient exceptions via a class method named `transient_exceptions`, like so:

```
  class RefundJob < ApplicationJob
»   def self.transient_exceptions
»     [ Stripe::RateLimitError ]
»   end

    def perform(order_id)
      order = Order.find(order_id)
      Refunder.new.refund(order)
    end
  end
```

The middleware should go in `lib/sidekiq_middleware/server/silence_transient_errors.rb`. It will call `yield`, then examine the exception raised. If the exception's class is in the middleware's list *or* the list provided by the job class, it'll wrap it in `IgnorableExceptionSinceSidekiqWillRetry`. Otherwise, the exception will be reraised (which will presumably trigger your error catcher).